ESPECIALIDAD DE SEGUNDO TRAMO EN TÉCNICAS DE APOYO CIS – TIC

SISTEMAS OPERATIVOS

TRABAJO FINAL

Sgt1 Jonatan D. Chamizo

Sgt1 Hugo Ondicol

<u>PRIMER EJERCICIO:</u> Contador Regresivo (Usando while y Variables)

Crear un contador regresivo interactivo que se detenga al llegar a cero, usando un bucle while:

1. Pide al usuario que introduzca un número inicial para la cuenta regresiva y guarda la entrada en una variable llamada CONTADOR (usando read).

2. Utiliza un bucle while que se ejecute mientras el valor de CONTADOR sea mayor que cero.

3. Dentro del bucle, imprime el valor actual de CONTADOR.

4. Implementa un sleep 1 para pausar la ejecución por un segundo.

5. Decrementa el valor de la variable CONTADOR en 1 en cada iteración.

6. Cuando el bucle termine, imprime "¡Lanzamiento completado!".

Estructuras Clave: while, Variables, Operación Aritmética.

Abrir | *ejercicio1contador.sh ~/Documentos | Guardar

```
1 #!/bin/bash
2 echo "INTRODUCE UN NÚMERO INICIAL PARA LA CUENTA REGRESIVA: "
3 read contador
4
5 while [ $contador -gt 0 ]
6 do
7 echo $contador
8 sleep 1
9 contador=$((contador-1))
10
11 done
12 echo "FINALIZA CUENTA ATRÁS"
13
```

hondara@hondara-VirtualBox: ~/Documentos
~/Documentos

```
./ejercicio1contador.sh: línea 5: [: -gt: se esperaba un operador unario
FINALIZA CUENTA ATRÁS
hondara@hondara-VirtualBox:~/Documentos$ gedit ejercicio1contador.sh

** (gedit:39013): WARNING **: 09:29:44.862: Could not load Peas repository: Typelib file for namespace 'Peas', versio
n '1.0' not found

** (gedit:39013): WARNING **: 09:29:44.862: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk',
version '1.0' not found
hondara@hondara-VirtualBox:~/Documentos$ ./ejercicio1contador.sh
INTRODUCE UN NÚMERO INICIAL PARA LA CUENTA REGRESIVA:
10
10
9
8
7
6
5
4
3
2
1
FINALIZA CUENTA ATRÁS
hondara@hondara-VirtualBox:~/Documentos$
```

Explicación EJERCICIO 1:

```
#!/bin/bash

echo "INTRODUCE UN NÚMERO INICIAL PARA LA CUENTA REGRESIVA:"

read contador

while [ $contador -gt 0 ]do

echo $contador

sleep 1

contador=$((contador-1))

done

echo "¡LANZAMIENTO COMPLETADO!"
```

Empieza con #!/bin/bash, que le dice al sistema que este archivo se va a ejecutar como un programa de Bash, o sea, un guion de comandos de Linux.

El comando echo "INTRODUCE UN NÚMERO INICIAL PARA LA CUENTA REGRESIVA:" muestra un mensaje en pantalla pidiendo un número.

La línea read contador lee el número que escribe el usuario y lo guarda en la variable contador.

Luego comienza un bucle while [$contador -gt 0], que significa “mientras el contador sea mayor que 0”.

Dentro del bucle, echo $contador muestra el número actual, sleep 1 hace una pausa de 1 segundo, y contador=$((contador-1)) reduce el contador en 1 cada vez. Esto se repite hasta que el número llega a 0.

Cuando el bucle termina, el script muestra el mensaje “¡LANZAMIENTO COMPLETADO!”, indicando que la cuenta regresiva ha terminado.

<u>SEGUNDO EJERCICIO:</u> Creador de Archivos Múltiples (Usando for y Variables) El objetivo es usar un bucle for para crear una serie de archivos con nombres secuenciales.

1. Define una variable llamada PREFIJO con el valor "Informe_".

2. Define una variable llamada NUM_ARCHIVOS con el valor 5.

3. Utiliza un bucle for para iterar desde 1 hasta el valor de NUM_ARCHIVOS.

4. En cada iteración, usa el comando touch para crear un archivo cuyo nombre sea la combinación de PREFIJO y el número de iteración.

Ejemplo de nombres: Informe_1.txt, Informe_2.txt, etc.

5. Al finalizar, imprime un mensaje que confirme cuántos archivos se crearon.

Estructuras Clave: for, Variables, touch

Abrir ejercicio2.sh ~/Documentos Guardar

```
#!/bin/bash

PREFIJO="Informe_"
NUM_ARCHIVOS=5

for (( i=1; i<=NUM_ARCHIVOS; i++ ))
do
touch "${PREFIJO}${i}.txt"
done
echo "Se crearon $NUM_ARCHIVOS archivos con el prefijo $PREFIJO"
```

hondara@hondara-VirtualBox: ~/Documentos ~/Documentos

```
version '1.0' not found
hondara@hondara-VirtualBox:~/Documentos$ gedit ejercicio2.sh

** (gedit:39689): WARNING **: 10:33:53.993: Could not load Peas repository: Typelib file for namespace 'Peas', versio
n '1.0' not found

** (gedit:39689): WARNING **: 10:33:53.993: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk',
version '1.0' not found
hondara@hondara-VirtualBox:~/Documentos$ chmod 777 ejercicio2.sh
hondara@hondara-VirtualBox:~/Documentos$ ./ejercicio2.sh
Se crearon 5 archivos con el prefijo Informe_
hondara@hondara-VirtualBox:~/Documentos$ ls -s
total 32
0 Informe_1.txt  0 Informe_4.txt          4 ejercicio2.sh             4 prueba_for.sh    4 pruebas.sh
0 Informe_2.txt  0 Informe_5.txt          4 ejercicio2armultiples.sh  4 prueba_if.sh
0 Informe_3.txt  4 ejercicio1contador.sh  4 ejercicio3.sh             4 prueba_while.sh
hondara@hondara-VirtualBox:~/Documentos$
hondara@hondara-VirtualBox:~/Documentos$
hondara@hondara-VirtualBox:~/Documentos$
hondara@hondara-VirtualBox:~/Documentos$
hondara@hondara-VirtualBox:~/Documentos$
hondara@hondara-VirtualBox:~/Documentos$
hondara@hondara-VirtualBox:~/Documentos$
hondara@hondara-VirtualBox:~/Documentos$
```

Explicación EJERCICIO 2:

```
#!/bin/bash

PREFIJO="Informe_"

NUM_ARCHIVOS=5

for (( i=1; i<=NUM_ARCHIVOS; i++ ))

do

touch "${PREFIJO}${i}.txt"

done

echo "Se crearon $NUM_ARCHIVOS archivos con el prefijo $PREFIJO"
```

Empieza con #!/bin/bash, que le dice al sistema que este archivo se va a ejecutar como un programa de Bash.

Después, PREFIJO="Informe_" guarda en una variable la palabra base que tendrán todos los archivos, en este caso “Informe_”.

Luego, NUM_ARCHIVOS=5 guarda el número total de archivos que se quieren crear, que aquí son cinco.

La línea for ((i=1; i<=NUM_ARCHIVOS; i++)) inicia un bucle, que es como decirle a la computadora: “repite lo siguiente varias veces”; aquí empieza en 1, sigue mientras i sea menor o igual que 5, y en cada vuelta suma 1.

La palabra do marca el comienzo de las órdenes que se repetirán dentro del bucle.

Después, touch es el comando que crea el archivo vacío si no existe y touch "${PREFIJO}${i}.txt" crea un archivo nuevo usando el texto guardado en PREFIJO más el número actual de la vuelta, así que va formando nombres como Informe_1.txt, Informe_2.txt y así hasta Informe_5.txt

La palabra done indica que ahí termina el bloque de instrucciones que se repite.

Por último, echo "Se crearon $NUM_ARCHIVOS archivos con el prefijo $PREFIJO" muestra en pantalla un mensaje final confirmando cuántos archivos se crearon y cuál fue el prefijo usado.

TERCER EJERCICIO: Verificación de Directorios (Combinando for, if y Variables)

Este script combina el bucle for con una comprobación condicional (if) para verificar el estado de una lista de directorios.

1. Define una variable llamada DIRECTORIOS que contenga una lista de nombres separados por espacios (algunos existentes y otros no, ej: "src data /etc/mis_docs").

2. Utiliza un bucle for para recorrer cada elemento de la variable DIRECTORIOS.

3. Dentro del bucle, utiliza una estructura if para comprobar si el directorio actual existe (usa el test -d).

4. Si el directorio existe, imprime un mensaje verde de éxito: "Directorio [nombre] ENCONTRADO."

5. Si no existe, imprime un mensaje rojo de advertencia: "Directorio [nombre] NO EXISTE."

Estructuras Clave: for, if/else, Variables, Test (-d)

Abrir | ejercicio3.sh ~/Documentos | Guardar

```bash
#!/bin/bash

DIRECTORIOS="scr data /etc/mis_docs_inexistentes /home"

for DIR in $DIRECTORIOS
do

if [ -d "$DIR" ]; then
echo -e "\e[32mDirectorio $DIR ECONTRADO. \u2714\e[0m"

else
echo -e "\e[31mDirectorio $DIR NO EXISTE. \u2718\e[0m"

fi
done
```

hondara@hondara-VirtualBox: ~/Documentos
~/Documentos

```
** (gedit:39447): WARNING **: 10:17:20.426: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk',
version '1.0' not found
hondara@hondara-VirtualBox:~/Documentos$ gedit ejercicio2.sh

** (gedit:39475): WARNING **: 10:18:06.720: Could not load Peas repository: Typelib file for namespace 'Peas', versio
n '1.0' not found

** (gedit:39475): WARNING **: 10:18:06.720: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk',
version '1.0' not found
hondara@hondara-VirtualBox:~/Documentos$ gedit ejercicio3.sh

** (gedit:39484): WARNING **: 10:18:19.889: Could not load Peas repository: Typelib file for namespace 'Peas', versio
n '1.0' not found

** (gedit:39484): WARNING **: 10:18:19.889: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk',
version '1.0' not found
hondara@hondara-VirtualBox:~/Documentos$ sudo chmod 777 ejercicio3.sh
hondara@hondara-VirtualBox:~/Documentos$ ./ejercicio3.sh
Directorio scr NO EXISTE. ✘
Directorio data NO EXISTE. ✘
Directorio /etc/mis_docs_inexistentes NO EXISTE. ✘
Directorio /home ECONTRADO. ✔
hondara@hondara-VirtualBox:~/Documentos$
```

Explicación EJERCICIO 3:

```
#!/bin/bash

DIRECTORIOS="scr_data /etc/mis_docs_inexistentes /home"

for DIR in $DIRECTORIOS

do

if [ -d "$DIR" ]; then

echo -e "\e[32mDirectorio $DIR ECONTRADO. \u2714\e[0m"

else

echo -e "\e[31mDirectorio $DIR NO EXISTE. \u2718\e[0m"

fi

done
```

Empieza con #!/bin/bash, que le dice al sistema que este archivo se va a ejecutar como un programa de Bash.

DIRECTORIOS="scr_data /etc/mis_docs_inexistentes /home" guarda en una variable una lista de rutas de directorios que se quieren comprobar.

Después, for DIR in $DIRECTORIOS inicia un bucle que recorrerá cada uno de esos directorios uno por uno. La palabra do indica que empiezan las instrucciones que se repetirán para cada directorio.

Luego, if [-d "$DIR"] comprueba si el directorio actual existe en el sistema (-d significa “es un directorio”).

Si existe, se ejecuta echo -e "\e[32mDirectorio $DIR ECONTRADO. \u2714\e[0m", que muestra un mensaje en color verde diciendo que el directorio fue encontrado junto con un símbolo de corrección positiva.

Si no existe, entra en else y ejecuta echo -e "\e[31mDirectorio $DIR NO EXISTE. \u2718\e[0m", mostrando el mensaje en color rojo con una “X” indicando que no existe.

Finalmente, fi cierra la condición if y done termina el bucle for, después de haber revisado todos los directorios de la lista.